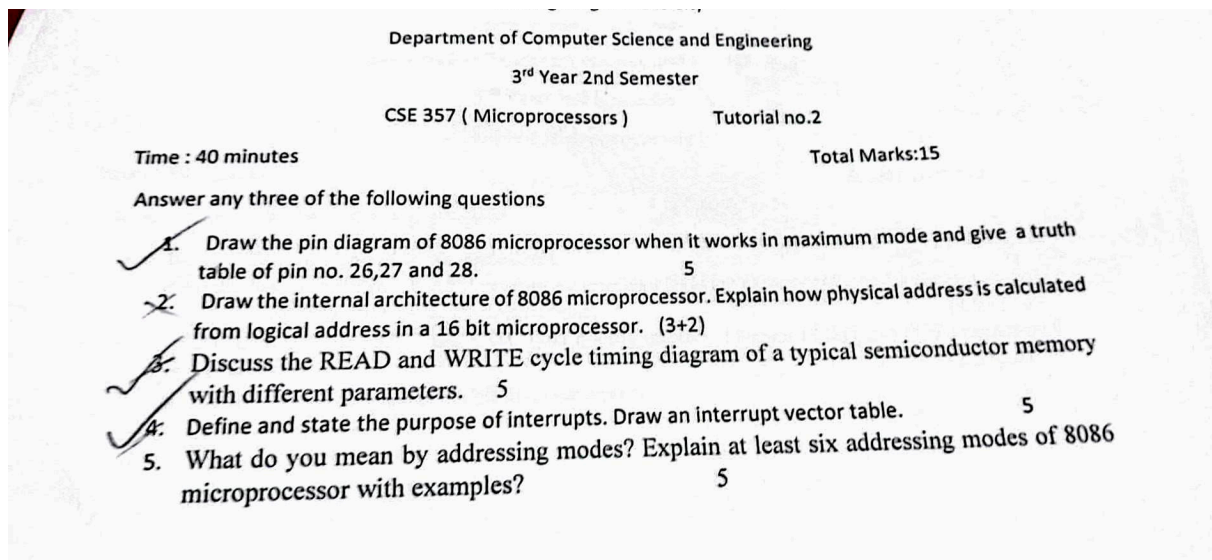




MP Tutorial 2 solve

1. Pin Diagram of 8086 Microprocessor in Maximum Mode

The **8086 microprocessor** can operate in **minimum mode** (single processor system) or **maximum mode** (multiprocessor system). In **maximum mode**, additional control signals are used to support multiprocessor configurations.

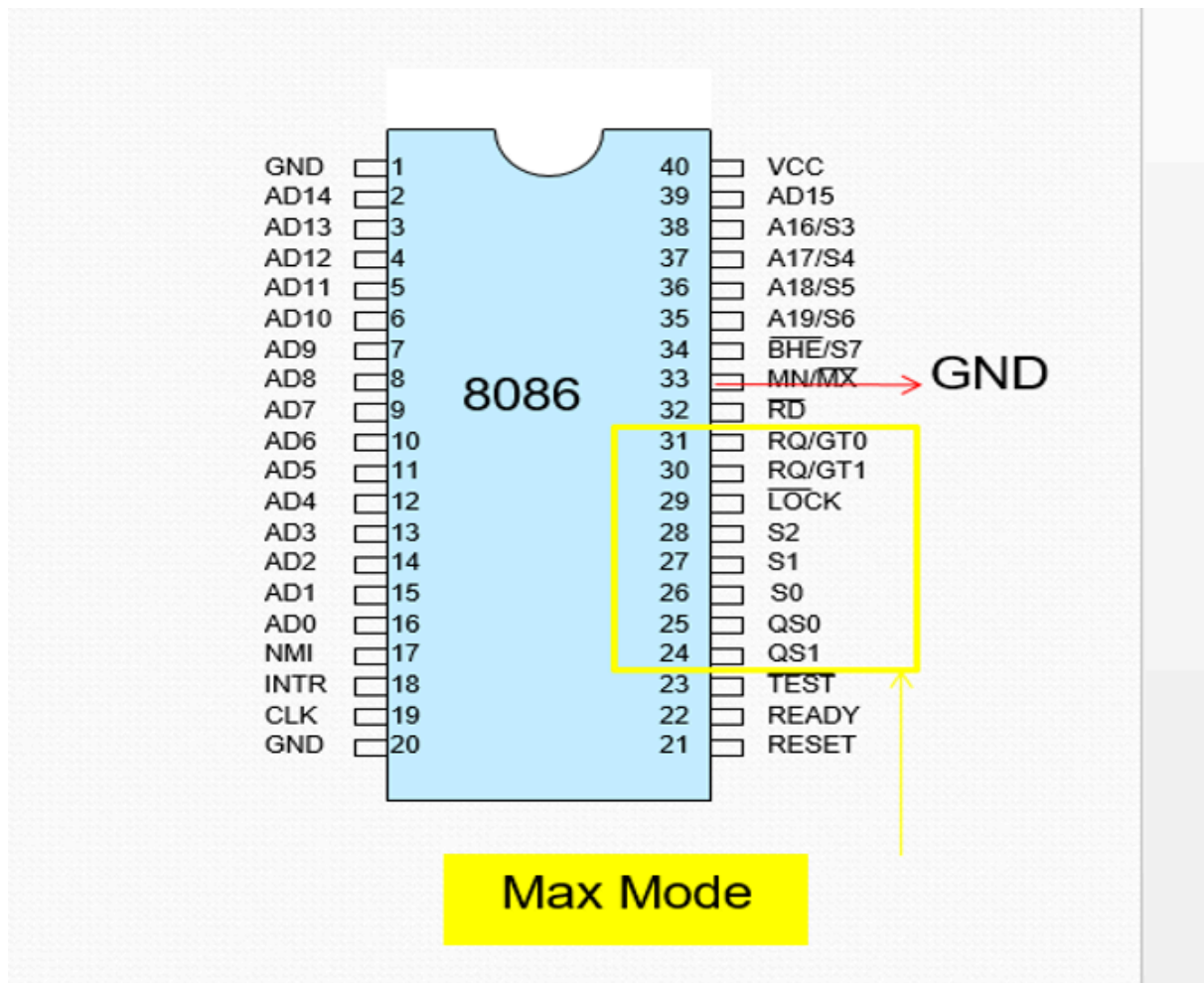
The 8086 microprocessor operates in **maximum mode** when the **MN/MX** pin is set to **low (0V)**. In this mode, the processor interacts with a **bus controller (8288)** to manage system operations.

The **pin diagram** of the **8086 in maximum mode** consists of **40 pins**, categorised into:

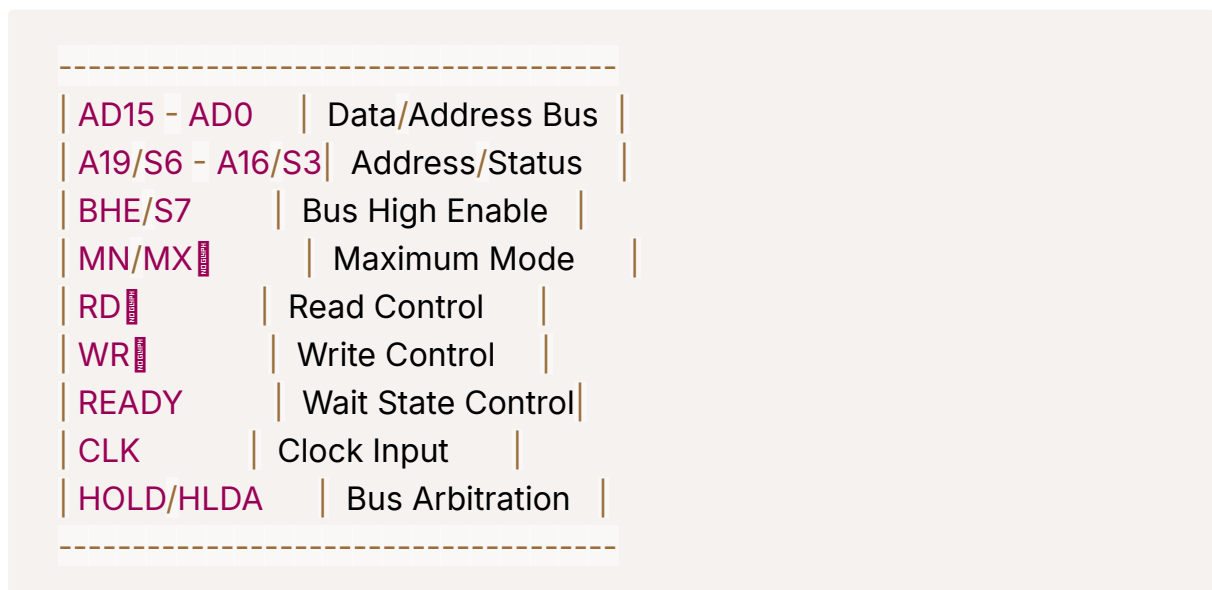
- **Address/Data Bus (AD0–AD15)**
- **Status and Control Signals**
- **Interrupt and DMA Signals**
- **Clock and Reset Signals**
- **Power Supply and Ground**

A detailed **pin diagram** should be drawn showing **MN/MX** = **0** to indicate maximum mode operation.

Pin Diagram of 8086 (Maximum Mode)



(Draw the following diagram in your answer sheet)



Truth Table for Pins 26, 27, and 28

Pin Number	Pin Name	Function in Maximum Mode
28	S2	Status line, used to indicate the processor cycle type.
27	S1	Works with S2 and S0 to define the current cycle.
26	S0	Works with S1 and S2 to define operation.

The combination of **S2**, **S1**, and **S0** determines the type of operation being performed.

$\overline{S_0}, \overline{S_1}, \overline{S_2}$
Pin 26, 27, 28 (Output)

$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$	Status
0	0	0	Interrupt Acknowledge
0	0	1	I/O Read
0	1	0	I/O Write
0	1	1	Halt
1	0	0	Opcode Fetch
1	0	1	Memory Read
1	1	0	Memory Write
1	1	1	Passive

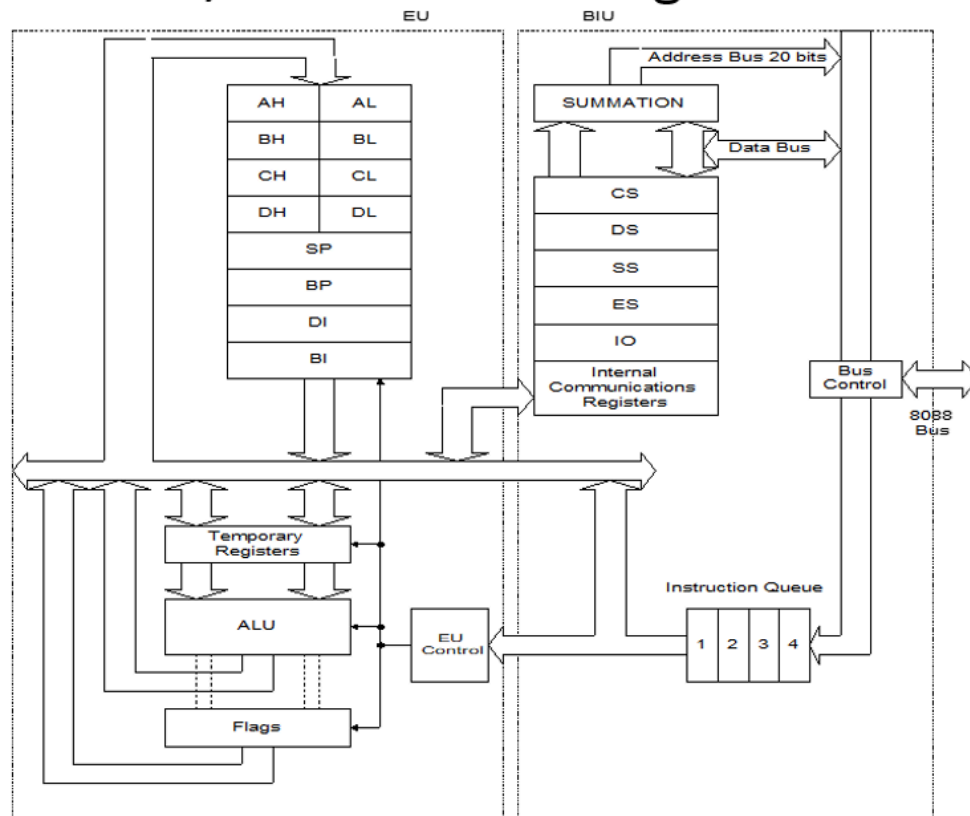
The diagram shows the 8086 microprocessor pinout. Pins 1 through 28 are listed on the left, and pins 29 through 40 are listed on the right. Pins 26, 27, and 28 are highlighted with a red box and labeled S0, S1, and S2 respectively.

S2	S1	S0	Function
0	0	0	Interrupt Acknowledge
0	0	1	Read I/O Port
0	1	0	Write I/O Port
0	1	1	Halt
1	0	0	Code Fetch
1	0	1	Read Memory
1	1	0	Write Memory
1	1	1	Passive(idle state)

2. Draw the internal architecture of the 8086 microprocessor. Explain how a physical address is calculated from a logical address in a 16-bit microprocessor.

Internal Architecture of 8086

8086/8088 Internal Organisation



Bus Interface Unit (BIU)

- Instruction Queue
- Address Calculation
- Segment Registers



The diagram illustrates the **internal organization of the Intel 8086/8088 microprocessor**, which consists of two major functional blocks:

1. Execution Unit (EU)

Handles the **execution of instructions** and includes:

◆ General Purpose Registers:

- **AX, BX, CX, DX** (split as AH/AL, BH/BL, CH/CL, DH/DL): Used for arithmetic, logic, and data movement.
- **SP (Stack Pointer), BP (Base Pointer)**: Stack operations.
- **SI (Source Index), DI (Destination Index)**: Used in string manipulation and addressing.

◆ Arithmetic Logic Unit (ALU):

- Performs **arithmetic and logical operations** (add, subtract, AND, OR, etc.).

◆ Temporary Registers:

- Hold intermediate data during instruction execution.

◆ Flags:

- Stores **status bits** like Zero, Carry, Sign, Overflow, etc.

◆ EU Control:

- Directs the internal operations of the EU, controlling data flow and instruction decoding.

2. Bus Interface Unit (BIU)

Responsible for **interfacing with memory and I/O devices**:

◆ Segment Registers:

- **CS (Code Segment), DS (Data Segment), SS (Stack Segment), ES (Extra Segment), IO (I/O related).**
- Used in **segment:offset** addressing to form a 20-bit physical address.

◆ Summation Unit:

- Adds segment base with offset to generate **20-bit physical address**.

◆ Instruction Queue:

- Prefetches instructions (up to 6 bytes for 8086, 4 bytes for 8088) to improve performance.
- **Pipeline-like behavior:** While EU executes, BIU fetches next instructions.

◆ Address/Data Buses:

- **Address Bus (20 bits):** Can access up to 1MB memory.
- **Data Bus:** 16 bits in 8086, 8 bits in 8088.

◆ Bus Control Logic:

- Handles **read/write cycles, interrupt acknowledgment, and control signals** to external devices.

How EU & BIU Work Together

- BIU **fetches instructions** and places them into the **instruction queue**.
- EU **reads instructions** from the queue and executes them.
- This **parallelism** improves speed and efficiency.

Key Difference Between 8086 and 8088

- **8086:** 16-bit data bus → faster data handling.
 - **8088:** 8-bit data bus → cheaper to implement, used in IBM PC.
-

Would you like a side-by-side comparison of 8086 vs 8088 as well?

The **8086 microprocessor** has a **16-bit architecture** with two primary functional units:

1. **Bus Interface Unit (BIU)**

- Handles memory and I/O access.
- Contains segment registers (CS, DS, SS, ES) and the **Instruction Queue (6 bytes)**.

2. **Execution Unit (EU)**

- Decodes and executes instructions.
- Contains registers (AX, BX, CX, DX), ALU, and control unit.

A **diagram** should be drawn showing both **BIU** and **EU**.

Calculation of Physical Address from Logical Address in a 16-bit Microprocessor (8086)

The **8086 microprocessor** uses a **segmented memory model** to calculate the physical address from a logical address. The memory is divided into **segments**, and each segment is addressed using a **segment:offset** pair.

1. Concept of Segmentation

- The **logical address** consists of two parts:
- **Segment Address (16-bit)**
- **Offset Address (16-bit)**
- The **Physical Address (20-bit)** is calculated by combining these two values.

2. Formula for Physical Address Calculation

The formula to calculate the **Physical Address (PA)** from the **Segment Address (SA)** and **Offset Address (OA)** is:

$$\text{Physical Address} = (\text{Segment Address} \times 16) + \text{Offset Address}$$

Since **multiplication by 16** (or shifting left by 4 bits) converts the segment address into a base address, adding the offset provides the actual memory location.

3. Example Calculation

Given:

- **Segment Address (SA) = 2000H**
- **Offset Address (OA) = 0500H**

Step 1: Convert Segment Address to Base Address

$$\text{Base Address} = 2000H \times 16 = 20000H$$

Step 2: Add the Offset

$$\text{Physical Address} = 20000H + 0500H = 20500H$$

So, the **physical address** is **20500H**.

4. Another Example

Given:

- Segment Address (SA) = 3F2A H
- Offset Address (OA) = 1234 H

Step 1: Convert Segment Address to Base Address

$$\text{Base Address} = 3F2AH \times 16 = 3F2A0H$$

Step 2: Add the Offset

$$\text{Physical Address} = 3F2A0H + 1234H = 404D4H$$

Thus, the **physical address** is 404D4H.

5. Advantages of Segmented Addressing

- ✓ **Efficient Memory Utilization:** Allows access to **1MB memory space** using 16-bit registers.
- ✓ **Modular Code Execution:** Different segments can be assigned for **code, data, and stack**, improving memory organization.
- ✓ **Easy Relocation:** Segments can be moved without changing the offset, supporting flexible memory management.

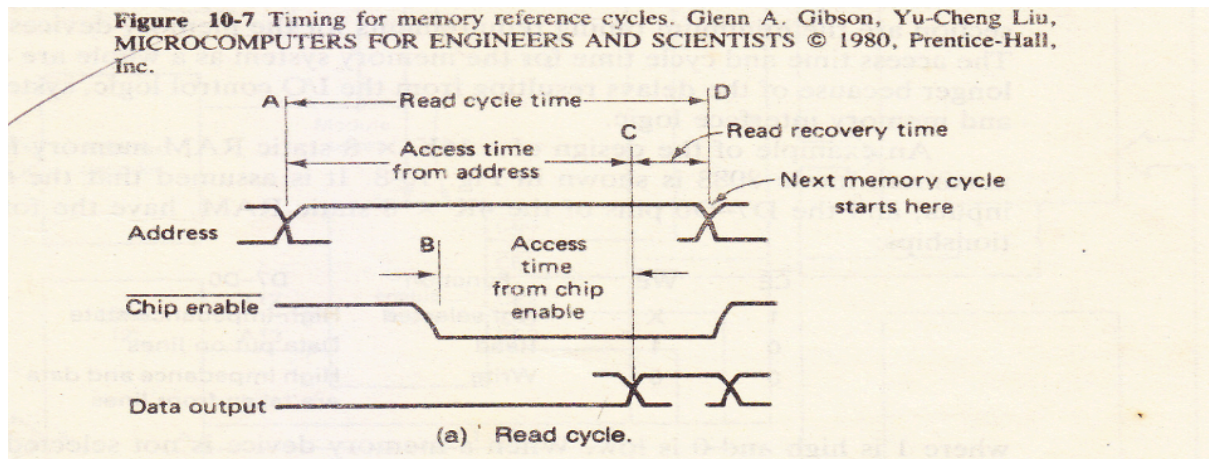
6. Conclusion

The **8086 microprocessor** uses segmentation to calculate **20-bit physical addresses** from **16-bit segment and offset addresses**. This method enables efficient memory management while maintaining compatibility with a **16-bit processor architecture**.

Would you like a diagram for better understanding? 🚀

3. Discuss the READ and WRITE cycle timing diagram of a typical semiconductor memory with different parameters.

Read Cycle Timing Diagram



1. Read Cycle (First Diagram)

This is the process of reading data from memory.

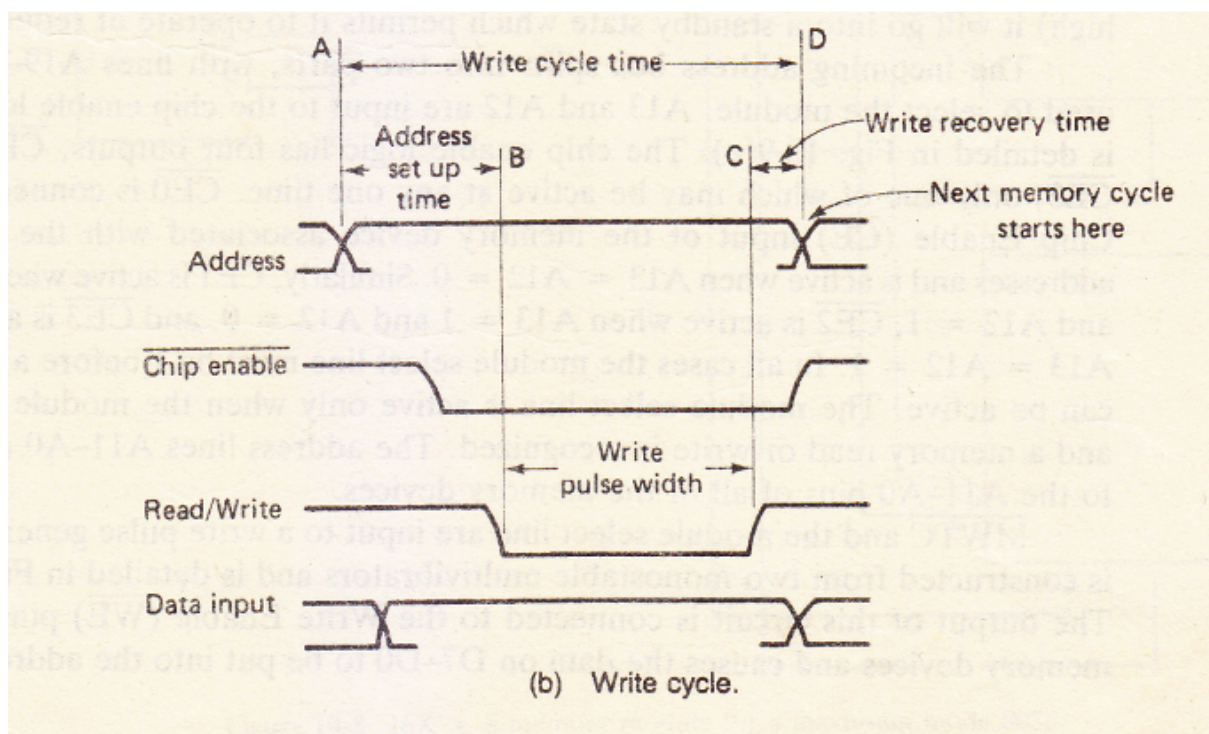
Signals Involved:

- **Address:** Specifies the memory location to be accessed.
- **Chip Enable (CE):** Activates the memory chip.
- **Data Output:** Shows when valid data is available.
- **Timing Points:**
 - **A → D: Read Cycle Time** – Total duration for one memory read cycle.
 - **A → C: Access Time from Address** – Time from when the address is valid to when valid data is available.
 - **B → C: Access Time from Chip Enable** – Time from chip being enabled to data availability.
 - **C → D: Read Recovery Time** – Time required after data is read before starting the next cycle.

Sequence:

1. Address is placed on the bus.
2. Chip Enable goes low to activate memory.
3. After access time, valid data appears on the data bus.
4. At the end of the cycle, chip enable goes high, ending the read.
5. System waits for read recovery before initiating another cycle.

Write Cycle Timing Diagram



2. Write Cycle (Second Diagram)

This is the process of writing data into memory.

Signals Involved:

- **Address:** Target memory location.
- **Chip Enable (CE):** Activates memory chip.
- **Read/Write (R/W):** Low indicates write operation.
- **Data Input:** Data to be written.

- **Timing Points:**

- **A → D: Write Cycle Time** – Total time for the write operation.
- **A → B: Address Setup Time** – Minimum time address must be stable before write starts.
- **B → C: Write Pulse Width** – Duration write enable signal is active.
- **C → D: Write Recovery Time** – Time needed after write completes before the next operation.

Sequence:

1. Address is set and held stable.
2. Chip enable goes low.
3. R/W line goes low (indicating write).
4. Data is placed on the bus.
5. After the write pulse, chip enable goes high.
6. Write recovery period follows before another cycle can begin.

4. Define and state the purpose of interrupts. Draw an interrupt vector table.

Definition of Interrupts:

An **interrupt** is a signal that temporarily stops the execution of a program and allows the processor to handle an urgent task.

Purpose of Interrupts:

- **Efficient CPU usage:** CPU does not waste time checking for events.
- **Handling I/O operations:** Allows the CPU to interact with external devices.
- **Error handling:** Detects critical errors (e.g., divide by zero).
- **Multitasking:** Supports switching between multiple tasks.

Interrupt Vector Table (IVT) of 8086:

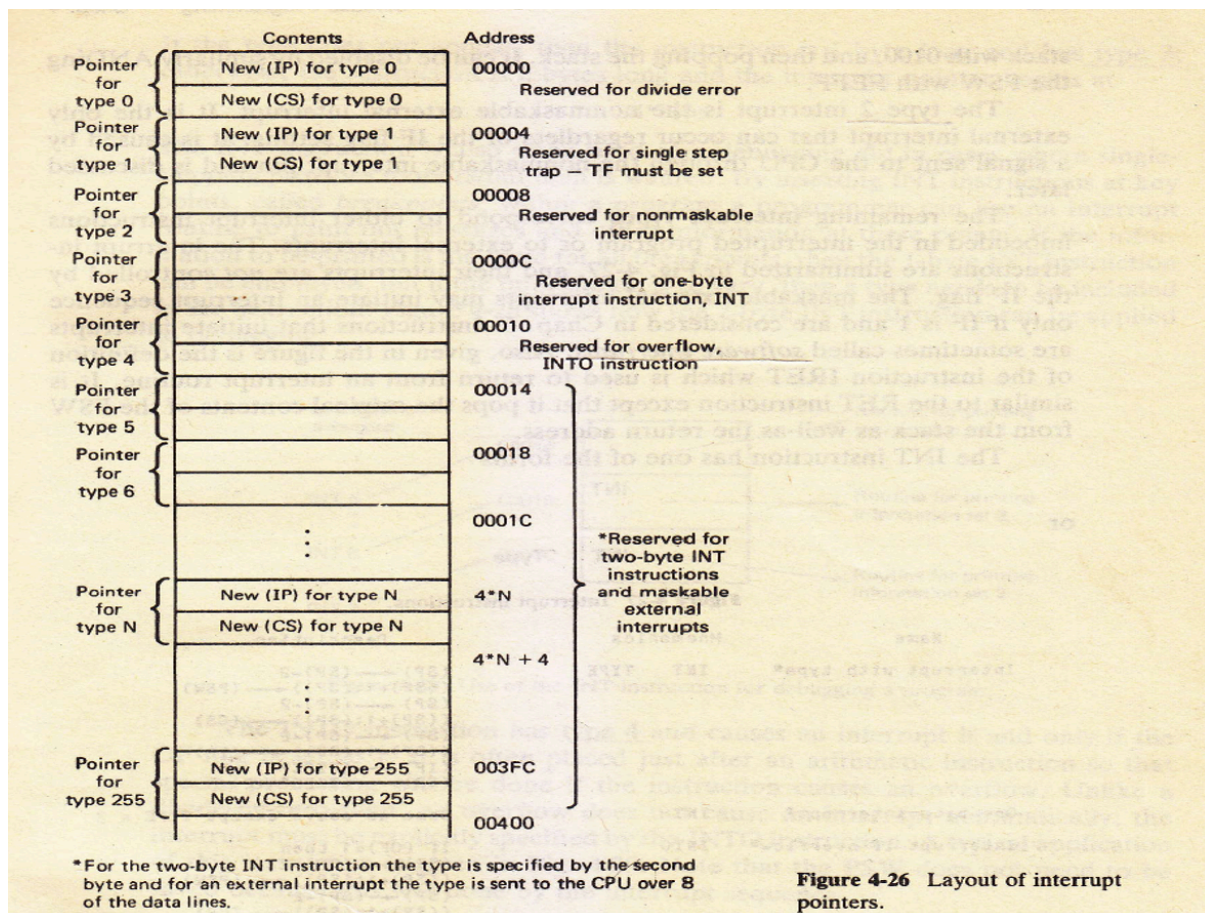


Figure 4-26 Layout of interrupt pointers.

What Is This Table?

IVT used by the CPU to find the **starting address of the Interrupt Service Routine (ISR)** when an interrupt occurs. Each entry (also called a "vector") contains the address of the ISR in the form of:

- **IP (Instruction Pointer)**
- **CS (Code Segment)**

Each interrupt type has **4 bytes** in the table:

- **2 bytes for IP**
- **2 bytes for CS**

Structure Details:

- The table starts at **physical memory address 00000H**.
- Each interrupt type occupies **4 bytes**.
- The address of the ISR for **interrupt type N** is stored at:

- $\text{Address} = 4 * N$
- IP is stored at $4 * N$, CS is stored at $4 * N + 2$
- There are **256 possible interrupt types** (0 to 255).

Example Entries:

Interrupt Type	IP Address Location	CS Address Location	Total Bytes Used	Purpose
0	00000H	00002H	4	Divide-by-zero error
1	00004H	00006H	4	Single-step (debug)
2	00008H	0000AH	4	Non-maskable interrupt (NMI)
3	0000CH	0000EH	4	Breakpoint (INT 3)
4	00010H	00012H	4	Overflow (INTO instruction)
...	...	...	...	
255	003FCH	003FEH	4	Last interrupt type

Notes:

- The **INT N** instruction uses this table to find the ISR.
- The table must be initialized by the **BIOS or operating system** during boot.
- The CPU reads the **IP and CS from this table** and jumps to that location on an interrupt.

Would you like a code example showing how this table is used in assembly?

5. What do you mean by addressing modes? Explain at least six addressing

modes of the 8086 microprocessor with examples.

Definition of Addressing Modes:

Addressing modes define the way **operands (data) are accessed** in an instruction.

Six Addressing Modes of 8086:

1. Immediate Addressing Mode

- Operand is **directly given** in the instruction.
- **Example:** MOV AX, 1234H (Move immediate value 1234H to AX)

2. Register Addressing Mode

- Operand is in a **register**.
- **Example:** MOV BX, AX (Move value from AX to BX)

3. Direct Addressing Mode

- Address of the operand is given in the instruction.
- **Example:** MOV AX, [5000H] (Move value from memory location 5000H to AX)

4. Indirect Addressing Mode

- The **memory address** is stored in a register.
- **Example:**

```
MOV BX, 5000H
MOV AX, [BX] ; Access memory at 5000H using BX
```

5. Indexed Addressing Mode

- Address is calculated using **Base + Index Register**.
- **Example:** MOV AX, [SI+5000H]

6. Base-Relative Addressing Mode

- Address is calculated using **Base Register + Displacement**.
- **Example:** MOV AX, [BX+04H] (Fetch data from memory at BX+4)

Conclusion:

These answers cover all five questions from your **CSE 357 (Microprocessors) - Tutorial no.2** question paper. Let me know if you need **diagrams, explanations, or corrections!** 🚀

Types of Addressing Modes

Addressing Mode	Explanation	Example
Immediate	Operand is given directly.	MOV AL, 05H
Register	Operand is in a register.	MOV AX, BX
Direct	Memory address is specified.	MOV AX, [1234H]
Indirect	Address is in a register.	MOV AX, [BX]
Indexed	Offset is added to a base register.	MOV AL, [SI+10H]
Base Indexed	Combines base and index registers.	MOV AL, [BX+SI]

Describe briefly synchronous and asynchronous communication

Communication between a microprocessor and external devices (like memory, I/O devices) can be categorized as **synchronous** and **asynchronous**, based on how data transfer is coordinated.

Synchronous Communication

In synchronous communication, **data is transmitted between the microprocessor and peripheral devices using a shared clock signal**. Both sender and receiver operate in sync with this clock, ensuring that data transfer happens at precise intervals.



Synchronous communication is a method of data transmission in which the **transmitter and receiver share a common clock signal** to synchronize the transfer of data.

Characteristics:

- Uses a **clock signal** to synchronize data transmission.
- Data is transmitted in a **continuous stream**.
- Faster and more efficient for large data transfers.
- Requires precise timing, so both sender and receiver must be **synchronized**.

Example:

- **8086 Microprocessor's Bus Interface Unit (BIU) and Execution Unit (EU)** communicate synchronously.
- **I2C (Inter-Integrated Circuit)** and **SPI (Serial Peripheral Interface)** use synchronous communication.

Diagram:



Here, data bits (D1, D2, D3...) are transmitted in sync with the clock pulses.

Asynchronous Communication

In asynchronous communication, data is transmitted **without a clock signal**. Instead, **start and stop bits** are used to indicate the beginning and end of data transmission.



Asynchronous communication is a method of data transmission where **no common clock is shared** between the sender and the receiver. Data is sent in **individual units (bytes)** with **start and stop bits**.

Characteristics:

- No clock signal; uses **start and stop bits** to synchronize data.
- Suitable for **low-speed or irregular data transfer**.
- **Start bit** and **stop bit** are used to define each data frame.
- Slower than synchronous communication but **more flexible**

Example:

- **Serial communication via UART (Universal Asynchronous Receiver-Transmitter).**
- **RS-232 standard** used in old computer ports.
- **Keyboards and Mice** often use asynchronous communication.

Diagram:

Start | Data Bits (D1, D2, D3...) | Stop
| —D1—D2—D3—D4—D5—D6—D7— |

Here, the **start bit** signals the beginning of transmission, and the **stop bit** marks the end.

Key Differences:

Conclusion

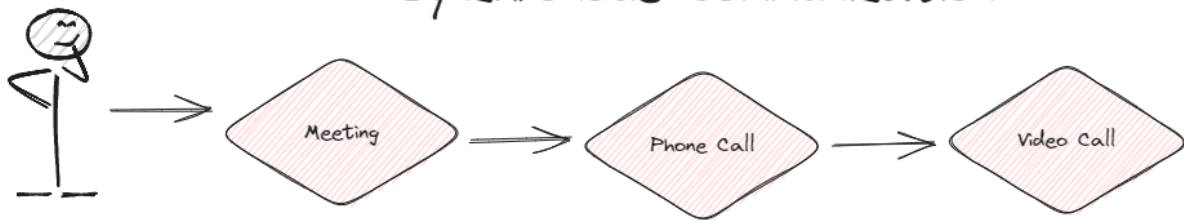
- **Synchronous communication** is ideal for high-speed, real-time data transfer where both devices are synchronized.
- **Asynchronous communication** is simpler and more flexible, making it suitable for long-distance or slower-speed communications.

Both methods are crucial in microprocessors, depending on the system requirements.

Key Differences:

Feature	Synchronous Communication	Asynchronous Communication
Clock Signal	Required	Not required
Speed	Faster	Slower
Complexity	More complex hardware	Simpler
Data Flow	Continuous	Character-based
Overhead	Lower	Higher (Start/Stop bits)
Examples	SPI, I ² C	UART, RS-232

Synchronous Communication



Asynchronous Communication

